

Software Requirements and Design Document

for

AVAL AI Reconciliation Engine

Prepared by

24I-0767 Rayyan Ahmad Sultan

24I-0581 Aryan Ali Khan

24I-0899 Safwan Shakir

SCOPE

CS-3004 Software Design & Analysis (CX01)

DATE

May 7, 2026

Table of Contents

1. Introduction

- 1.1 Purpose
- 1.2 Product Scope
- 1.3 Title
- 1.4 Objectives
- 1.5 Problem Statement

2. Overall Description

- 2.1 Product Perspective
- 2.2 Product Functions
- 2.3 List of Use Cases
- 2.4 Extended Use Cases
- 2.5 Use Case Diagram

3. Other Nonfunctional Requirements

- 3.1 Performance Requirements
- 3.2 Safety Requirements
- 3.3 Security Requirements
- 3.4 Software Quality Attributes
- 3.5 Business Rules
- 3.6 Operating Environment
- 3.7 User Interfaces

4. Domain Model

5. System Sequence Diagram

6. Sequence Diagram

7. Class Diagram

8. Package Diagram

9. Deployment Diagram

● Introduction

○ Purpose

This Software Requirements and Design Document (SRDD) specifies the software requirements and architectural design for AVAL — an AI-Powered Financial Reconciliation Engine developed as the semester deliverable for CS-3004 Software Design & Analysis. The document covers the complete scope of AVAL Version 1.0-BETA, a standalone Java 21/JavaFX 21 desktop application that automates the matching of internal company ledgers against external bank statements using a hybrid deterministic and semantic AI matching pipeline. This SRS describes the full system, including all twelve (12) use cases from client organization setup through audit-trail generation. Sub-systems such as the LangChain4j vectorization adapter and the PostgreSQL pgvector persistence layer are addressed in context of the larger architecture.

○ Product Scope

AVAL (Aval's AI Reconciliation Engine) is a desktop-class financial technology tool designed to solve the problem of manual, error-prone bank reconciliation faced by small-to-medium finance teams. The product occupies the niche of an intelligent, on-premise reconciliation workbench — distinct from cloud ERP modules in that all data and AI model inference remain entirely on the local machine, ensuring data sovereignty and compliance with financial privacy regulations.

The core value proposition of AVAL is its Hybrid Matching Engine, which combines deterministic Rule-Based Matching (for exact-amount, date-proximity transactions) with Semantic AI Vectorization (using the nomic-embed-text embedding model via Ollama and LangChain4j). This hybrid approach resolves the most common failure modes of traditional reconciliation tools: abbreviated merchant names, aggregated payments, and varying date conventions between ledgers and bank statements.

AVAL stores transaction embeddings as 768-dimensional vectors in a PostgreSQL 16 database extended with pgvector, enabling lightning-fast cosine similarity search without any cloud dependency. The full system is delivered as a single Maven project deployable with a two-command setup (Docker Compose + Maven Wrapper), targeting Windows, macOS, and Linux environments.

○ Title

AVAL — AI-Powered Financial Reconciliation Engine: Automating Internal Ledger-to-Bank-Statement Matching via a Hybrid Deterministic and Semantic AI Pipeline

The title captures the dual nature of the system: it is both a classical rules engine (deterministic, 100%-confidence exact matches) and an AI inference engine (probabilistic, semantic vector similarity). The immediate solution it provides is the elimination of manual, cell-by-cell reconciliation in spreadsheet applications.

○ Objectives

The major aims and goals targeted by AVAL are:

- Automate financial reconciliation for at least 95% of transaction pairs that share exact amounts and dates (Rule-Based Engine), reducing the manual workload to near-zero for high-confidence matches.
- Achieve an 80% reduction in total manual verification hours by routing borderline matches (70-94% confidence score) to a Human-in-the-Loop (HITL) review queue rather than rejecting them outright.

- Provide real-time financial visibility: a Finance Officer can see reconciliation status — Auto-Reconciled, Pending Review, or Unmatched — within seconds of data ingestion, not days.
- Eliminate dependency on vendor-specific bank statement formats by implementing an automated schema standardization layer (UC4) that maps any column structure to the AVAL universal financial schema.
- Support multi-source consolidation (UC9): enable N-way matching where multiple small ledger entries sum to a single bank deposit, a common pattern in retail e-commerce and invoice-aggregation scenarios.
- Generate a cryptographically auditable, immutable reconciliation record (UC12) for every finalized match, providing a defensible audit trail for external accountants and regulators.
- Run entirely on-premise with no cloud data egress, using locally-hosted Ollama models to preserve data sovereignty and comply with financial data privacy requirements.
- Achieve architectural quality sufficient for a Grade-A SDA submission: full adherence to GRASP (9 principles) and GoF (6 patterns) design standards as documented in the class diagram and sequence diagrams.

○ Problem Statement

Financial reconciliation — the process of confirming that a company's internal accounting ledger accurately reflects the transactions recorded by its bank — is one of the most labour-intensive, error-prone, and time-critical tasks in corporate finance. In a typical small-to-medium enterprise, a finance officer may spend 4-8 hours per reconciliation cycle manually cross-referencing hundreds or thousands of transactions in Microsoft Excel, looking for entries that "match" between two spreadsheets with fundamentally different data formats.

Traditional automated reconciliation tools address this problem by applying rigid exact-match rules: a transaction is considered matched if and only if its date and amount are identical in both datasets. This approach fails catastrophically in real-world conditions for at least three recurring reasons. First, merchant narratives diverge: the same payment appears as "AMZN Mktp US*1A2B3C" in the bank statement and "Amazon Web Services — November Invoice" in the ledger. Second, aggregated payments are common: three individual invoice payments of PKR 50,000 each may be consolidated into a single bank credit of PKR 150,000. Third, date conventions conflict: a bank may post transactions with a 2-3 business day value date delay relative to the ledger's transaction date.

AVAL directly addresses all three failure modes. Semantic Vectorization (UC5) encodes merchant names into high-dimensional mathematical vectors so that "AMZN" and "Amazon" map to nearby points in 768-dimensional space regardless of abbreviation. Multi-Source Consolidation (UC9) allows one bank entry to be reconciled against a group of ledger entries whose amounts sum within a configurable tolerance. The configurable 7-day date proximity window in MatchingConfig ensures that legitimate date-offset transactions are candidates for matching rather than being discarded as unmatched. The result is a feasible, semester-scope prototype that demonstrates that AI-powered reconciliation is achievable with standard open-source tooling at zero cloud cost.

● Overall Description

○ Product Perspective

AVAL is a new, self-contained desktop product rather than a component of an existing system. It does not integrate with or depend on any third-party ERP, CRM, or accounting platform in its Version 1.0 scope. The system is designed to be a standalone workbench that a Finance Officer or AVAL Engineer operates on a local machine.

The system interfaces with exactly two external infrastructure components, both of which are provisioned automatically via Docker Compose: (1) a PostgreSQL 16 database instance extended with the pgvector extension for vector similarity search, and (2) an Ollama inference server hosting the nomic-embed-text embedding model. Both components run as Docker containers on the same machine as the JavaFX application, communicating over a private Docker bridge network. There is no internet connectivity required during normal operation.

The product family context: AVAL is designed as Version 1 of an extensible platform. Version 2 (deferred) targets ERP JSON push integration, digital report certification via CryptoHashService, WhatsApp OCR for informal payment records, and full RBAC with organizational admin roles. These are noted as V2 deferred features in the class diagram notes.

○ Product Functions

AVAL provides the following major functional groups:

- Client & Workspace Management: Provision isolated multi-tenant reconciliation environments per client organization (UC1).
- Data Ingestion: Parse and ingest internal ledgers in .xlsx format (UC2) and external bank statements in .pdf format (UC3) using Apache POI and Apache PDFBox respectively.
- Schema Standardization: Automatically normalize disparate column headers and data types from any ingested file into the AVAL universal financial schema (UC4).
- AI Vectorization: Convert transaction narratives into 768-dimensional semantic embeddings via the nomic-embed-text model (UC5), stored in PostgreSQL as vector(768) columns.
- Hybrid Matching Engine: Execute a two-pass probabilistic matching pipeline (UC6) combining Rule-Based exact matching (100% confidence) and Semantic Cosine Distance search (70-99% confidence).
- Human-in-the-Loop Review: Present AI-suggested match hypotheses to a Finance Officer for approval or rejection (UC7).
- Manual Override: Allow an administrator to force-link any two unmatched transactions with an audit justification (UC8).
- Multi-Source Consolidation: Match a group of ledger entries whose amounts sum to a single bank transaction within a configurable tolerance (UC9).
- Anomaly Detection: Identify suspicious transactions including duplicates, statistical outliers (>3 standard deviations), and weekend postings (UC10).
- Report Generation: Produce a verified reconciliation summary with an unresolved-items gate that prevents report export when pending hypotheses remain (UC11).
- Audit Trail: Maintain an immutable, timestamped history of all finalized reconciliation records accessible to the Finance Officer (UC12).

○ List of Use Cases

The following table summarises all twelve use cases with their primary actors, levels, and implementation status as of May 2026:

UC #	Name	Primary Actor	Scope Level	Status
UC1	Create Client Organization Profile	AVAL Engineer	User-Goal	✓ Done
UC2	Ingest Client Internal Ledger	AVAL Engineer	User-Goal	✓ Done
UC3	Import External Financial Data	AVAL Engineer	User-Goal	✓ Done
UC4	Standardize Financial Schema	System	Sub-function	✓ Done
UC5	Execute Semantic Vectorization	AI Engine	Sub-function	✓ Done
UC6	Run Probabilistic Matching Engine	AI Engine	User-Goal	✓ Done
UC7	Review AI-Suggested Matches	Finance Officer	User-Goal	✓ Done
UC8	Force Manual Reconciliation	Admin	User-Goal	✓ Done
UC9	Perform Multi-Source Consolidation	Finance Officer	User-Goal	✓ Done
UC10	Identify Financial Anomalies	System	Sub-function	✓ Done
UC11	Generate Verified Reconciliation Report	Finance Officer	User-Goal	✓ Done
UC12	Create & Maintain Transaction History	Finance Officer	User-Goal	✓ Done

○ Extended Use Cases

The following fully-dressed use case specifications describe the complete interaction flow, pre/post conditions, and exception paths for each of the twelve use cases:

Use Case 1: Create Client Organization Profile

Use Case Name	Create Client Organization Profile
Scope	AVAL Financial Reconciliation Platform
Level	User-Goal
Primary Actor	AVAL Engineer
Stakeholders & Interests	• AVAL Engineer: Wants to quickly and securely set up a new isolated workspace for a client without technical friction. • Client: Wants guaranteed data privacy so their financial records are kept strictly separate from other companies.
Main Success Scenario	1. Navigate to the client management dashboard and click Create New Workspace. 2. System displays the new client configuration form. 3. Enter the client company's details, unique bank account numbers, and specific accounting rules. 4. Click the provision workspace button.

	5. System validates the entered data to ensure no duplicate profiles or invalid bank formats exist. 6. System sets up a new secure, isolated database workspace for the client. 7. System displays confirmation message: "Workspace Created Successfully!".
Extensions	• The system detects a duplicate company name/ID and displays an error. The Actor is asked to enter a new or update the existing profile. • The system detects an invalid account length or format. It highlights the field in red and waits until corrected.
Pre-Conditions	• The AVAL Engineer is authenticated into the system with administrative/setup privileges.
Post-Conditions	• A secure, isolated workspace is created in the database for the new client, ready to receive and process ledger and bank data.

Use Case 2: Ingest Client Internal Ledger

Use Case Name	Ingest Client Internal Ledger
Scope	AVAL Financial Reconciliation Platform
Level	User-Goal
Primary Actor	AVAL Engineer
Stakeholders & Interests	• AVAL Engineer: Wants a seamless and error-free way to upload the client's raw financial data into the system. • Client: Wants their internal company records accurately transferred without data loss or corruption.
Main Success Scenario	1. Navigate to the data ingestion module and select the option to upload the internal ledger. 2. System asks the user to select the target Client Workspace and browse for the ledger file. 3. Select the specific client and upload the ledger file (.xlsx format via ExcelLedgerParser). 4. System scans the uploaded file to verify it is a supported format and checks for basic structural integrity. 5. System ingests the raw data and temporarily stores it in the pre-processing queue. 6. System displays a success message along with a brief preview of the raw rows and columns.
Extensions	• The system detects an unsupported file type or unreadable data. It displays an error and aborts the upload, asking the user to upload a valid file. • The uploaded file is too large for a single ingestion batch. The system displays a warning.
Pre-Conditions	• The AVAL Engineer is authenticated into the platform. • A target client workspace has already been successfully created (UC1 complete).

Post-Conditions	• The ledger file is saved in the system and is ready to be standardized in UC4.
------------------------	--

Use Case 3: Import External Financial Data

Use Case Name	Import External Financial Data
Scope	AVAL Financial Reconciliation Platform
Level	User-Goal
Primary Actor	AVAL Engineer
Stakeholders & Interests	• AVAL Engineer: Wants to quickly upload external bank statements to provide the matching engine with the required external data. • Client: Wants assurance that their sensitive bank statements are securely uploaded to their specific workspace.
Main Success Scenario	1. Navigate to the data ingestion module and select the option to upload external bank data. 2. System asks the user to select the target Client Workspace and browse for the external statement file. 3. Select the specific client and upload the bank statement (.pdf format via PDFBankStatementParser). 4. System scans the uploaded file to verify it is a supported format and checks for basic structural integrity. 5. System ingests the external data and temporarily stores it in the pre-processing queue. 6. System displays a success message along with a brief preview of the extracted data.
Extensions	• The system detects an unsupported file type or unreadable data. It displays an error and aborts the upload. • The system detects that some important bank columns are missing from the file. It flags the file as invalid and asks the user to upload a complete statement.
Pre-Conditions	• The AVAL Engineer is authenticated into the platform. • A target client workspace has already been successfully created.
Post-Conditions	• The external bank statement is loaded into the system, ready for the automated standardization step (UC4).

Use Case 4: Standardize Financial Schema

Use Case Name	Standardize Financial Schema
Scope	AVAL Financial Reconciliation Platform
Level	Sub-function
Primary Actor	System (IngestionService)

Stakeholders & Interests	• AI Matching Engine: Needs organized, normalized data to run AI matching properly. • AVAL Engineer: Wants the system to automatically fix different bank formats to avoid manual Excel editing.
Main Success Scenario	1. System retrieves the raw dataset from the temporary landing zone. 2. System scans the dataset to identify varying column headers (e.g., "Date", "Trans Date", "Value Date"). 3. System maps the identified headers to the universal AVAL financial schema (valueDate, amount, narrative, transactionType). 4. System cleanses data types within the columns, converting string amounts to BigDecimal and raw dates to LocalDate. 5. System saves the newly standardized dataset into the core reconciliation database (standardized_ledger / standardized_bank tables). 6. System flags the dataset status as "STANDARDIZED" and ready for the vectorization phase (UC5).
Extensions	• If the system finds a column name it does not recognise, it skips that specific column and logs a warning for the AVAL Engineer. • If a column meant for numbers contains text, the system removes that specific row to keep the dataset clean and logs an error warning.
Pre-Conditions	• The raw internal ledger or external bank file has been successfully ingested (UC2 or UC3 complete).
Post-Conditions	• The data is standardized into one uniform format and securely stored as StandardizedTransaction records, ready for AI vectorization.

Use Case 5: Execute Semantic Vectorization

Use Case Name	Execute Semantic Vectorization
Scope	AVAL Financial Reconciliation Platform
Level	Sub-function
Primary Actor	AI Engine (LangChain4jVectorizationEngine)
Stakeholders & Interests	• Finance Officer: Needs cryptic and code-mixed transaction narratives accurately understood by the system. • System Architect: Ensures high-dimensional representation allows for efficient similarity comparison.
Main Success Scenario	1. System retrieves a batch of StandardizedTransaction records from the PostgreSQL database (status = STANDARDIZED). 2. Engine performs pre-processing: lower-case conversion, punctuation removal, and text serialization in the format "TYPE AMOUNT NARRATIVE". 3. System invokes the locally-hosted Ollama instance running nomic-embed-text to parse the financial intent of the text. 4. LLM performs semantic parsing, recognising that "K-Elec" and "K-Electric Bill" represent the same entity regardless of shorthand.

	5. A 768-dimensional semantic vector is generated for each transaction description. 6. Vectors are indexed in the PostgreSQL database (vector(768) column) for cosine similarity retrieval and the dataset status is updated to VECTORIZED.
Extensions	• 3a. LLM API is unavailable: The system retries the connection; if it fails, it flags the description for basic keyword matching and subsequent manual review.
Pre-Conditions	• Raw financial data has been ingested and normalized into a standardized schema (UC4 complete).
Post-Conditions	• Textual descriptions are transformed into numerical vectors stored in the database, ready for the matching engine.

Use Case 6: Run Probabilistic Matching Engine

Use Case Name	Run Probabilistic Matching Engine
Scope	AVAL Financial Reconciliation Platform
Level	User-Goal
Primary Actor	AI Engine (HybridMatchingEngine)
Stakeholders & Interests	• Finance Officer: Interested in achieving the 80% reduction in manual verification workload. • Business Owner: Desires real-time financial visibility through high-speed automated balancing.
Main Success Scenario	1. System applies exact-match logic via RuleBasedMatchingEngine for transactions with identical amounts and dates within the 7-day tolerance window (100% confidence, MatchType.EXACT_RULE). 2. For remaining unmatched entries, the engine performs multi-factor semantic matching via SemanticMatchingEngine: evaluating date proximity, amount tolerances, and cosine distance between stored vectors. 3. System utilises world knowledge encoded in nomic-embed-text (e.g., recognising bank fee patterns) to improve matching accuracy. 4. A confidence score (0.0-1.0) is calculated for every potential transaction pair. 5. Matches exceeding the 0.95 threshold (MatchingConfig.AUTO_CONFIRM_THRESHOLD) are automatically reconciled as MatchHypothesis with status AUTO_RECONCILED. 6. System updates the reconciliation status in the centralized PostgreSQL database and flags remaining hypotheses for review.
Extensions	• 4a. Low Confidence Match (0.70-0.94): The system flags the pair as a "Suggested Match" (status PENDING_REVIEW) and queues for UC7 human review. • 4b. No Potential Match Found (<0.70 confidence): The transaction remains in the "Unmatched" pool, eligible for UC8 Force Manual Reconciliation or UC9 Multi-Source Consolidation.

Pre-Conditions	• Semantic vectorization (UC5) is complete for the current data batch.
Post-Conditions	• Transactions are categorized as Auto-Reconciled, Suggested Match (pending review), or Unmatched.

Use Case 7: Review AI-Suggested Matches

Use Case Name	Review AI-Suggested Matches
Scope	AVAL Financial Reconciliation Platform
Level	User-Goal
Primary Actor	Finance Officer
Stakeholders & Interests	• Finance Officer: Needs to maintain "Human-in-the-Loop" control over ambiguous transactions. • External Auditor: Requires a clear justification for why specific ambiguous items were linked.
Main Success Scenario	1. The Finance Officer logs into the system and navigates to the Manual Check panel. 2. System loads all pending MatchHypothesis records (status = PENDING_REVIEW) in a sortable table. 3. Finance Officer inspects each hypothesis, reviewing the confidence score, XAI explanation text (getXAICueText()), and the two transaction details side-by-side. 4. Finance Officer clicks Approve or Reject for each hypothesis. 5. On Approve: System calls ReconciliationService.confirmHypothesis(), creating an immutable ReconciliationRecord and updating hypothesis status to APPROVED. 6. On Reject: System calls ReconciliationService.rejectHypothesis(), updating status to REJECTED. Transaction re-enters the unmatched pool. 7. System updates the progress bar and checks if all hypotheses are resolved, enabling the Proceed to Report button when complete.
Extensions	• Finance Officer rejects a hypothesis that should have been approved: the transaction re-enters the unmatched pool and can be manually force-linked via UC8. • No pending hypotheses remain: the panel automatically enables the Proceed to Report action.
Pre-Conditions	• The matching engine (UC6) has completed and generated at least one PENDING_REVIEW hypothesis. • The Finance Officer is authenticated with the ACCOUNTANT role.
Post-Conditions	• All PENDING_REVIEW hypotheses are resolved. The workspace is ready for report generation (UC11) or anomaly review (UC10).

Use Case 8: Force Manual Reconciliation

Use Case Name	Force Manual Reconciliation
Scope	AVAL Financial Reconciliation Platform
Level	User-Goal
Primary Actor	Administrator (ADMIN role)
Stakeholders & Interests	• Administrator: Needs an override capability to reconcile transactions that the AI engine could not match automatically. • Finance Officer: Wants unmatched transactions resolved before the reconciliation cycle can be closed.
Main Success Scenario	1. Administrator navigates to the Manual Check panel and views the Unmatched Transactions section. 2. System displays two tables: Unmatched Ledger entries and Unmatched Bank entries from MainUIContext. 3. Administrator selects one ledger entry and one bank entry to force-link. 4. Administrator enters a mandatory justification note in the text field. 5. Administrator clicks the Force Link button. 6. System calls ReconciliationService.forceReconcile(ledger, bank, user, justification), creating a ReconciliationRecord with MatchType.FORCE_OVERRIDE. 7. System removes the force-linked transactions from the unmatched tables and adds the record to the reconciled history. 8. System checks if all unmatched transactions are resolved, updating the completion bar accordingly.
Extensions	• Only one entry selected from either table: the Force Link button remains disabled. • Justification field is blank: the Force Link button remains disabled. The system enforces mandatory justification. • ReconciliationService throws an exception: System displays an error label and re-enables the Force Link button.
Pre-Conditions	• The matching engine (UC6) and human review (UC7) have completed. • At least one unmatched ledger-bank transaction pair exists. • The user is authenticated with the ADMIN role.
Post-Conditions	• The selected transactions are force-reconciled with an audit note. The ReconciliationRecord with isManualOverride()==true is persisted to the database.

Use Case 9: Perform Multi-Source Consolidation

Use Case Name	Perform Multi-Source Consolidation
Scope	AVAL Financial Reconciliation Platform
Level	User-Goal
Primary Actor	Finance Officer

Stakeholders & Interests	• Finance Officer: Needs to handle the common scenario where multiple small ledger invoices are batched into one large bank deposit. • Business Owner: Wants the reconciliation engine to handle aggregated payment scenarios without manual splitting.
Main Success Scenario	1. Finance Officer navigates to the Multi-Source Consolidation panel in the Manual Check view. 2. System displays the Unmatched Bank transactions table (target) and the Unmatched Ledger transactions table (multi-select source). 3. Finance Officer selects one target bank transaction. 4. Finance Officer multi-selects multiple ledger entries whose amounts should sum to the bank transaction amount. 5. System shows a live running sum of selected ledger amounts in the consolidation sum label. 6. Finance Officer sets the tolerance percentage (default 2%) and clicks Consolidate. 7. System calls <code>ReconciliationService consolidateMultiSource()</code> which computes whether the ledger group sum falls within <code>[bankAmt - tolerance, bankAmt + tolerance]</code>. 8. If within tolerance: System creates one <code>ReconciliationRecord</code> per grouped ledger entry (<code>MatchType.FORCE_OVERRIDE</code>) and removes them from the unmatched pool. 9. If outside tolerance: System logs a <code>CONSOLIDATION_VARIANCE</code> Anomaly and displays it in the anomaly panel.
Extensions	• No bank entry selected: Consolidate button remains disabled. • No ledger entries selected: Consolidate button remains disabled. • Sum is outside tolerance: A <code>CONSOLIDATION_VARIANCE</code> anomaly is appended to the anomaly list and displayed to the Finance Officer.
Pre-Conditions	• The matching engine (UC6) has completed. • At least one unmatched bank transaction and multiple unmatched ledger entries exist. • The Finance Officer is authenticated.
Post-Conditions	• Multiple ledger entries are consolidated against a single bank transaction. <code>ReconciliationRecords</code> are persisted, and the transactions are removed from the unmatched pool.

Use Case 10: Identify Financial Anomalies

Use Case Name	Identify Financial Anomalies
Scope	AVAL Financial Reconciliation Platform
Level	Sub-function
Primary Actor	System (<code>AnomalyDetectionEngine</code>)
Stakeholders & Interests	• Finance Officer: Wants automatic flagging of suspicious transactions without having to manually inspect every unmatched entry.

	• External Auditor: Requires documented evidence that anomalous transactions were identified and reviewed before closing the reconciliation.
Main Success Scenario	1. System retrieves all unmatched ledger and bank transactions after the matching phase completes. 2. AnomalyDetectionEngine runs three detection passes: (1) Exact Duplicate detection within each dataset, (2) Statistical Outlier detection (>3 standard deviations from mean amount), (3) Weekend/Sunday posting detection. 3. Each detected anomaly is categorized as DUPLICATE, OUTLIER, WEEKEND_POSTING, or CONSOLIDATION_VARIANCE. 4. System creates typed Anomaly objects with category, description, and references to the affected StandardizedTransaction records. 5. Anomalies are stored in MainUIContext and displayed in the anomaly panel of the Manual Check view. 6. Finance Officer reviews anomalies and dismisses them after inspection. 7. Dismissal of all anomalies enables the Proceed to Report button (in combination with resolved hypotheses and unmatched transactions).
Extensions	• No anomalies detected: The anomaly panel is hidden and anomalies are marked as dismissed automatically. • Finance Officer dismisses anomalies without fully reviewing: The system completes the dismissal as-is; the audit note on ReconciliationRecords provides the trail.
Pre-Conditions	• The matching engine (UC6) has completed and unmatched transaction lists are populated.
Post-Conditions	• All anomalies are identified, categorized, and presented to the Finance Officer. The reconciliation workspace is ready for report generation once anomalies are dismissed.

Use Case 11: Generate Verified Reconciliation Report

Use Case Name	Generate Verified Reconciliation Report
Scope	AVAL Financial Reconciliation Platform
Level	User-Goal
Primary Actor	Finance Officer
Stakeholders & Interests	• Finance Officer: Needs a formal, exportable summary of the reconciliation outcome to submit to management or auditors. • External Auditor: Requires a verifiable document listing all matched, unresolved, and anomalous items.
Main Success Scenario	1. Finance Officer navigates to the Report panel and clicks Generate Report. 2. System calls ReportService.generateReconciliationReport() with the finalized lists of reconciled records, unmatched transactions, pending hypotheses, and anomalies. 3. System validates that no pending hypotheses remain; if they do, an UnresolvedItemsException is thrown and an error dialog is displayed.

	4. System compiles a summary including total transactions, matched count, unmatched count, anomaly count, and per-record audit notes. 5. System exports the report as a timestamped CSV file (and optionally PDF) to the local filesystem. 6. System displays a success dialog with the export path and allows the Finance Officer to open the file directly.
Extensions	• <code>UnresolvedItemsException</code>: If any <code>PENDING_REVIEW</code> hypotheses remain in the queue, the report generation is blocked. The system displays an error dialog listing the unresolved items and returns the user to the review queue.
Pre-Conditions	• The Finance Officer is authenticated with the <code>ACCOUNTANT</code> role. • All <code>MatchHypothesis</code> records have been resolved (<code>APPROVED</code>, <code>REJECTED</code>, or <code>FORCE_RECONCILED</code>). • All anomalies have been dismissed.
Post-Conditions	• A verified reconciliation report CSV (and optional PDF) is exported to the local filesystem. The report serves as the official close document for the reconciliation cycle.

Use Case 12: Create and Maintain Verified Transaction History

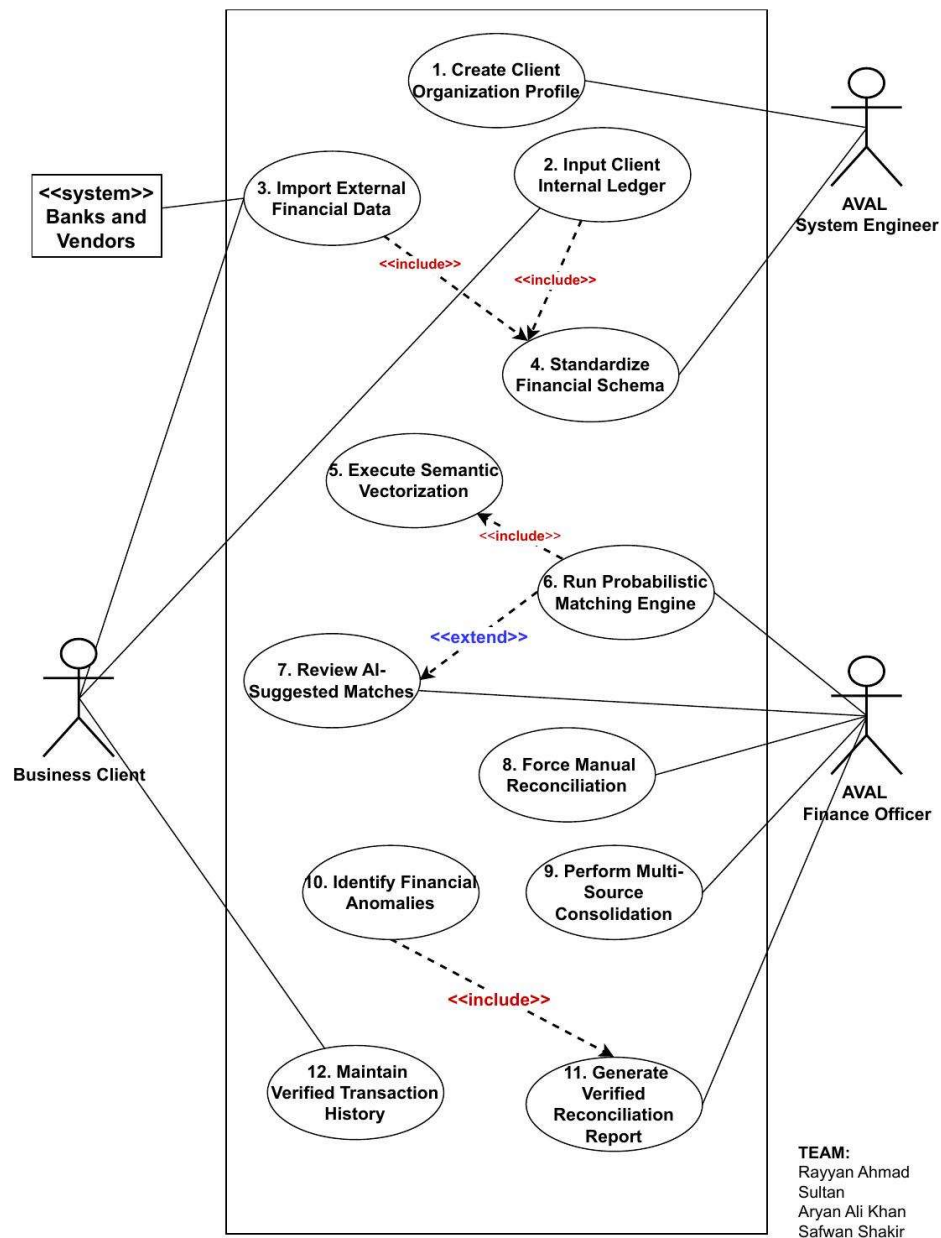
Use Case Name	Create and Maintain Verified Transaction History
Scope	AVAL Financial Reconciliation Platform
Level	User-Goal
Primary Actor	Finance Officer
Stakeholders & Interests	• Finance Officer: Needs to access historical reconciliation data for month-over-month trend analysis and compliance reporting. • External Auditor: Requires an immutable, timestamped history of all finalized reconciliation decisions.
Main Success Scenario	1. Finance Officer navigates to the Finance History panel (<code>FinanceController</code>). 2. System queries the <code>DataStore</code> for all <code>ReconciliationRecord</code> entries associated with the current workspace. 3. System displays the history in a sortable <code>TableView</code> showing: matched amount, approval date, sealing user, match type, and audit note. 4. Finance Officer can filter by date range, match type (<code>EXACT_RULE</code>, <code>AI_PROBABILISTIC</code>, <code>FORCE_OVERRIDE</code>), or sealed-by user. 5. Finance Officer can request a semantic proof log for any AI-matched record (displaying the XAI justification text). 6. Finance Officer can export the full history as a certified CSV for external use.
Extensions	• No reconciliation history exists for the workspace: System displays an empty state message and prompts the user to complete the reconciliation workflow first. • <code>DataStore</code> connection error: System displays a connection error dialog and logs the exception.

Pre-Conditions	• The Finance Officer is authenticated. • At least one reconciliation cycle has been completed and ReconciliationRecords have been persisted to the database.
Post-Conditions	• The Finance Officer has access to a complete, filterable, exportable audit trail of all finalized transaction matches for the selected client workspace.

○ Use Case Diagram

The following diagram illustrates the relationships between actors (AVAL Engineer, Finance Officer, Administrator, AI Engine, System) and all twelve use cases:

AVAL's Financial Reconciliation Platform



● Other Nonfunctional Requirements

○ Performance Requirements

The AVAL system must meet the following performance requirements under standard operating conditions (Intel Core i5 or equivalent, 8 GB RAM, SSD storage):

- Vectorization throughput: The LangChain4jVectorizationEngine must process a batch of 500 StandardizedTransaction records within 120 seconds when using the local nomic-embed-text Ollama model, equating to a maximum of 240ms per transaction for embedding generation.
- Cosine similarity search: The pgvector ORDER BY embedding <=> ?::vector query against a standardized_ledger table of up to 10,000 rows must return the top-3 nearest neighbours within 200ms, including JDBC round-trip time.
- Rule-based matching: The RuleBasedMatchingEngine must process all exact-match candidates across a dataset of up to 2,000 transaction pairs within 2 seconds.
- UI responsiveness: All JavaFX Task<T> background operations (ingestion, vectorization, matching) must update the UI progress indicator within 500ms of a status change. The UI thread must never be blocked for more than 100ms.
- Report generation: ReportService.generateReconciliationReport() must complete and write the CSV file within 10 seconds for a workspace containing up to 5,000 reconciliation records.
- Startup time: The JavaFX application (excluding Docker Compose infrastructure startup) must reach the authentication screen within 5 seconds on the target hardware.

○ Safety Requirements

AVAL handles sensitive financial data and must enforce the following safety requirements to prevent data loss, corruption, or unauthorized exposure:

- Immutable audit trail: Once a ReconciliationRecord is written to the database, it must not be modifiable or deletable through any application interface. All writes to the reconciliation_records table use INSERT-only operations; no UPDATE or DELETE statements are issued against this table.
- Unresolved-items gate: ReportService enforces an UnresolvedItemsException gate that prevents report export when any PENDING_REVIEW MatchHypothesis exists in the workspace. This prevents the generation of a reconciliation report that omits unresolved discrepancies.
- Data validation at ingestion: All ingested files are validated for structural integrity before any data is written to the database. Invalid files are rejected at the DocumentParser.validate() stage and an error is surfaced to the user.
- Local-only AI inference: All embedding generation is performed using a locally-hosted Ollama model. No financial transaction data is transmitted to any external API or cloud service at any point during normal operation.
- HikariCP connection pooling: Database connections are managed through a HikariCP pool with configurable max pool size and connection timeout, preventing resource exhaustion under concurrent operations.

○ Security Requirements

The following security requirements govern user authentication, data access control, and privacy within AVAL:

- Password hashing: All user passwords are hashed using BCrypt (bcrypt 0.4) with a work factor of 10 before storage. Plaintext passwords are never stored or logged.
- Role-Based Access Control (RBAC): The system enforces a two-role RBAC model via the UserRole enumeration. The ADMIN role may access Force Manual Reconciliation (UC8) functions; the ACCOUNTANT role is restricted to data ingestion, review, report generation, and history viewing.
- JDBC parameterization: All database queries in DataStore use PreparedStatement with parameterized inputs. No SQL query is constructed via string concatenation, eliminating SQL injection vulnerabilities.
- Workspace isolation: Each ClientOrganization workspace is assigned a unique UUID. All queries are scoped to the active workspaceId, preventing cross-client data access.
- Privacy-preserving AI: The system uses only locally-hosted embedding models (Ollama nomic-embed-text). No customer transaction narrative is transmitted to any third-party inference API.
- Session management: The active SystemUser and ReconciliationWorkspace are maintained in the MainUIContext singleton and cleared upon logout, preventing session data leakage between users.

○ **Software Quality Attributes**

AVAL is designed to exhibit the following measurable software quality attributes, implemented through deliberate application of GRASP and GoF design patterns:

- Maintainability: The Strategy Pattern (MatchingEngine interface with RuleBasedMatchingEngine, SemanticMatchingEngine, and HybridMatchingEngine implementations) ensures that a new matching algorithm can be introduced by adding a single class without modifying any existing code. The Template Method Pattern (DocumentParser<T> abstract class) provides the same extensibility for new file format parsers.
- Testability: Constructor injection is used throughout all service classes (IngestionService, ReconciliationService, ReportService), enabling mock objects to be injected in unit tests. Zero static singletons exist in the service layer (MainUIContext is the only singleton, confined to the UI layer). The test suite covers 25 test classes across domain, engine, persistence, service, and UI layers.
- Portability: The application is packaged as a Maven project using the Maven Wrapper (mvnw), ensuring reproducible builds without requiring a specific Maven version to be installed. The Docker Compose configuration ensures the PostgreSQL + pgvector + Ollama infrastructure is identically provisioned on Windows, macOS, and Linux.
- Reliability: The HikariCP connection pool with configurable pool size and timeout prevents database connection exhaustion. The UnresolvedItemsException gate in ReportService prevents the generation of incomplete reports, ensuring that only fully-verified data is exported.
- Low Coupling (GRASP): The MatchingEngine interface decouples ReconciliationService from any specific matching algorithm. The VectorizationEngine interface decouples the service layer from LangChain4j. The DataStore (Repository/DAO pattern) decouples all business logic from raw JDBC and pgvector SQL.
- High Cohesion (GRASP): IngestionService handles only UC2-UC4 (ingest, validate, standardize). ReportService handles only UC11-UC12 (report generation and history). ReconciliationService handles only UC5-UC10 (vectorize, match, review, force, consolidate, anomaly).

○ Business Rules

The following operating principles govern AVAL's matching behaviour and cannot be overridden at runtime without modifying MatchingConfig:

- **Auto-Confirm Threshold:** Any MatchHypothesis with a confidence score ≥ 0.95 (MatchingConfig.AUTO_CONFIRM_THRESHOLD) is automatically set to AUTO_RECONCILED without requiring human review.
- **Review Floor:** Any hypothesis with a score between 0.70 and 0.94 (MatchingConfig.REVIEW_FLOOR) is set to PENDING_REVIEW and routed to the Finance Officer's review queue.
- **Below-Floor Threshold:** Any hypothesis with a score below 0.70 is considered an unmatched transaction and does not generate a MatchHypothesis at all.
- **Date Tolerance Window:** The RuleBasedMatchingEngine accepts exact-amount matches where the two transaction dates differ by at most 7 calendar days (MatchingConfig.DATE_TOLERANCE_DAYS). This accommodates standard bank value-date posting delays.
- **Amount Tolerance:** A 1% amount tolerance (MatchingConfig.AMOUNT_TOLERANCE_PCT = 0.01) is applied for near-exact matches in the semantic engine to handle minor currency rounding differences.
- **Mandatory Force-Link Justification:** The Force Manual Reconciliation function (UC8) requires a non-blank justification string. The Force Link button is disabled until a justification is entered, ensuring every manual override is documented for audit purposes.
- **Multi-Candidate Limit:** The SemanticMatchingEngine retrieves a maximum of 3 nearest-neighbour candidates (MAX_CANDIDATES = 3) from the pgvector search for each query transaction, selecting the highest-confidence pair.
- **Report Generation Gate:** A reconciliation report may only be generated when all MatchHypothesis records in the workspace have a non-PENDING_REVIEW status. The ReportService enforces this via the UnresolvedItemsException.

○ Operating Environment

AVAL requires the following runtime environment for correct operation:

Component	Specification	Notes
JVM / Language	Java 21 (LTS)	OpenJDK 21+ required; Temurin or Adoptium recommended
UI Framework	JavaFX 21	Included via Maven pom.xml; FXML-based 9-view UI
Build Tool	Apache Maven (mvnw wrapper)	No separate Maven installation required
Database	PostgreSQL 16 + pgvector	Provisioned via Docker Compose; port 5432
AI Inference	Ollama (nomic-embed-text)	768-dim embeddings; provisioned via Docker Compose; port 11434
Containerization	Docker Compose	Required for PostgreSQL + Ollama; Docker Desktop recommended
AI Framework	LangChain4j 0.30.0	Dependency via Maven; no separate installation

Document Parsing	Apache PDFBox 3.0.2, Apache POI 5.2.5	PDF bank statements and XLSX ledger files
Operating System	Windows 10/11, macOS 12+, Ubuntu 22.04+	Cross-platform via JVM + Docker
RAM	Minimum 8 GB	16 GB recommended for large datasets
Storage	Minimum 5 GB free	For Ollama model (1.5 GB) + PostgreSQL data volume

○ User Interfaces

AVAL provides a JavaFX 21 graphical desktop application as its sole user interface. The GUI is styled with the custom fintech-light.css theme using the Jura variable font family. All views are defined as FXML files and managed by the MainUIContext singleton. The following nine views constitute the complete UI:

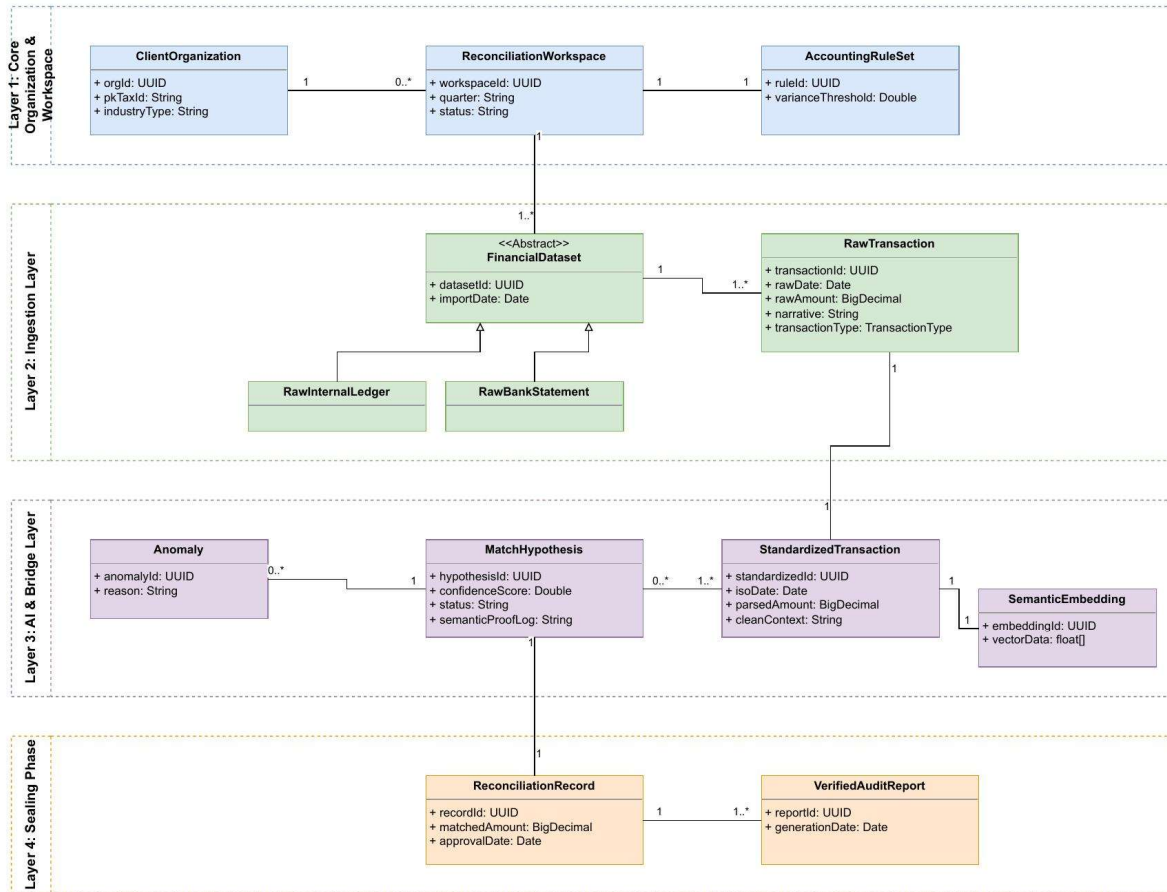
View (FXML)	Controller	Description
Splash.fxml	SplashController	Animated loading screen with AVAL branding displayed on application startup.
Auth.fxml	AuthController	Email/password login screen. Validates credentials against BCrypt-hashed passwords in the database.
Register.fxml	RegisterController	New user registration form for creating ADMIN or ACCOUNTANT accounts.
Registry.fxml	RegistryController	Client organization list and workspace creation form (UC1). Displays all registered ClientOrganization records.
Workspace.fxml	WorkspaceController	Main navigation hub. Contains the tab bar for Finance, Recon, Manual Check, and Report views.
Finance.fxml	FinanceController	Data ingestion panel (UC2, UC3) and reconciliation history viewer (UC12). Supports file upload for .xlsx ledgers and .pdf bank statements.
Recon.fxml	ReconController	Reconciliation execution panel. Triggers UC4 standardization, UC5 vectorization, and UC6 hybrid matching. Shows progress and match statistics.
ManualCheck.fxml	ManualCheckController	Human-in-the-Loop review panel (UC7), Force Manual Reconciliation (UC8), Multi-Source Consolidation (UC9), and Anomaly Review (UC10).
Report.fxml	ReportController	Report generation panel (UC11). Triggers ReportService and provides CSV/PDF export options. Displays the UnresolvedItemsException error when items remain.

All error messages are displayed as modal Alert dialogs. All long-running operations (ingestion, vectorization, matching, report generation) execute on daemon background threads using

JavaFX Task<T> to preserve UI thread responsiveness. Navigation between views is managed by WorkspaceController.showView() which uses SceneBuilder-compatible FXMLLoader.

● Domain Model

The Domain Model below shows the key domain concepts and their relationships within the AVAL system. It depicts the core entities (ClientOrganization, ReconciliationWorkspace, FinancialDataset, StandardizedTransaction, SemanticEmbedding, MatchHypothesis, ReconciliationRecord, SystemUser) and their multiplicities, as specified in the Domain and SSD artifact (Domain and SSD.pdf, Page 1).



● System Sequence Diagram

System Sequence Diagrams (SSDs) show the high-level system-level messages exchanged between actors and the AVAL system boundary for key use cases. These are extracted from the Domain and SSD.pdf artifact. The following SSDs are presented for UC1 through UC12, illustrating the system-level input/output messages for each major interaction.

UC1: Create Client Organization Profile SSD

Use Case 1

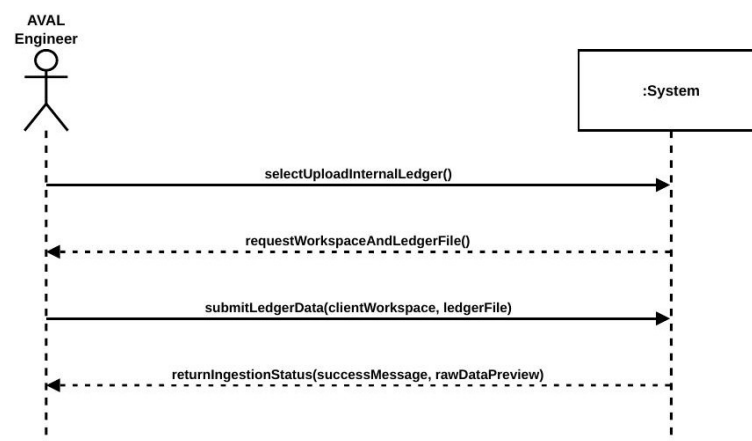
Create Client Organization Profile



UC2: Ingest Client Internal Ledger SSD

Use Case 2

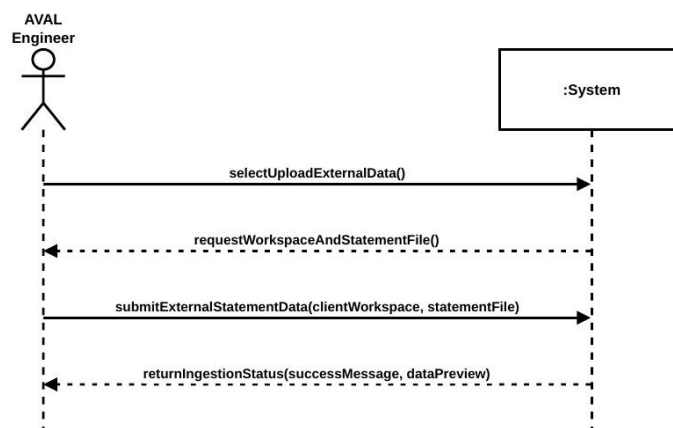
Ingest Client Internal Ledger



UC3: Import External Financial Data SSD

Use Case 3

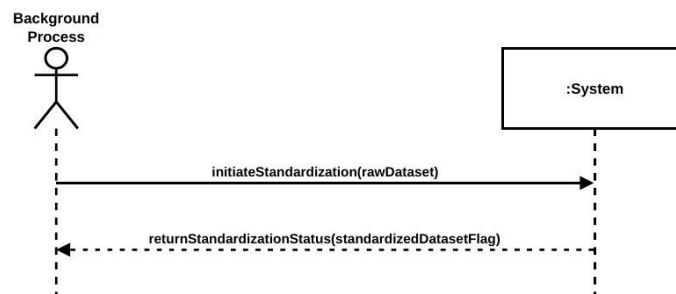
Import External Financial Data



UC4: Standardize Financial Schema SSD

Use Case 4

Standardize Financial Schema



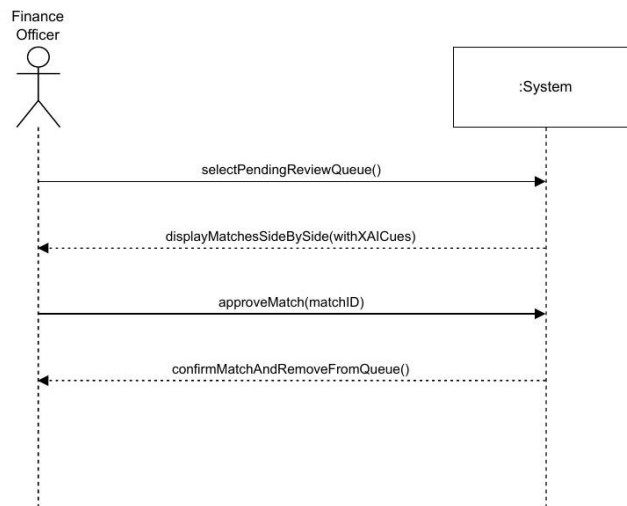
UC5: Execute Semantic Vectorization SSD

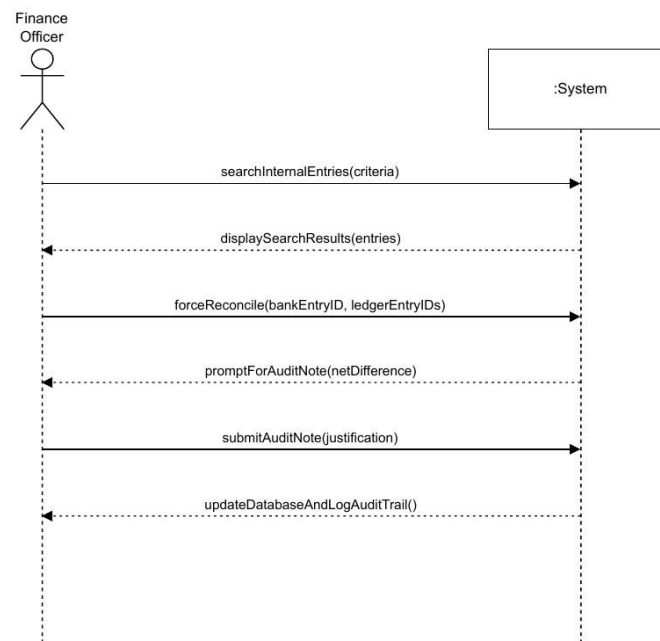


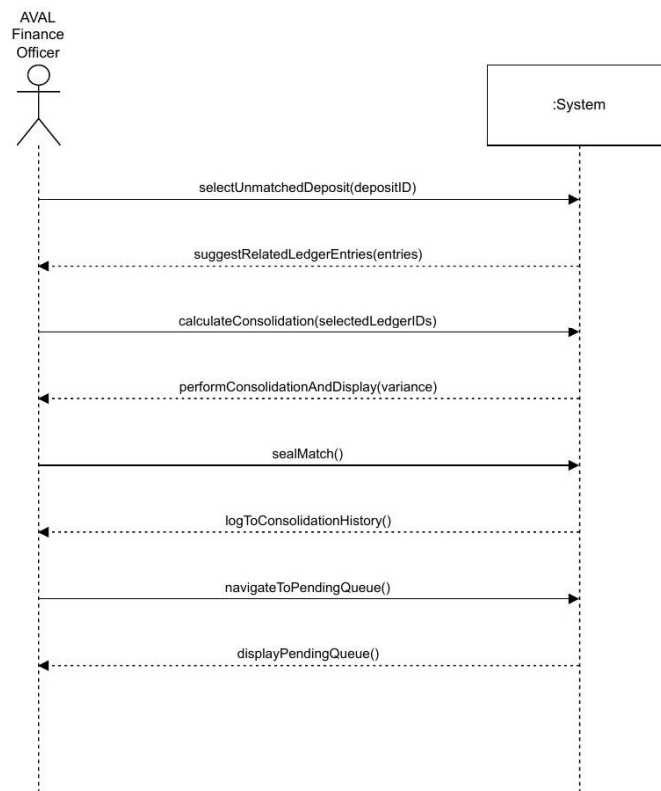
UC6: Run Probabilistic Matching Engine SSD

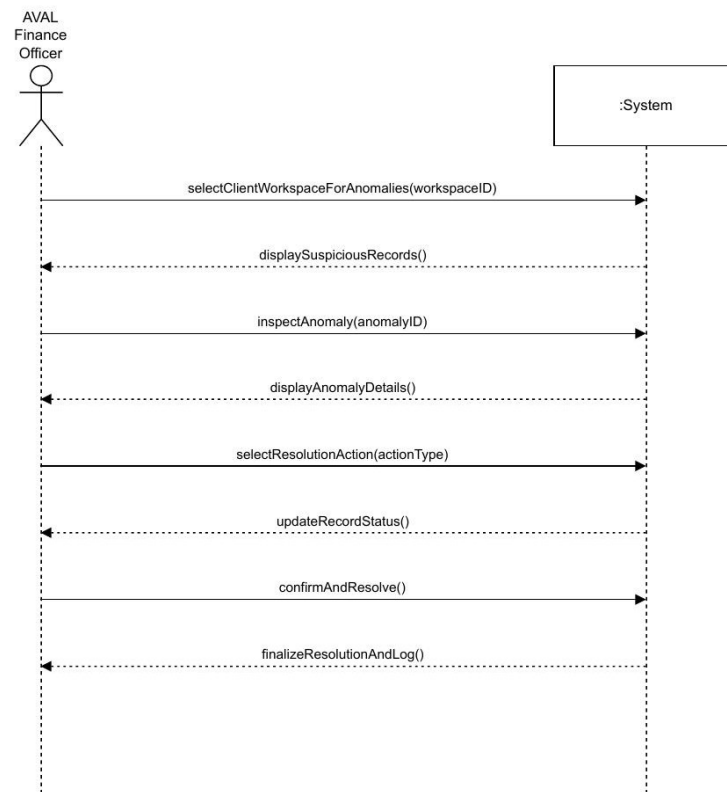


UC7: Review AI-Suggested Matches SSD

**UC8: Force Manual Reconciliation SSD**

**UC9: Perform Multi-Source Consolidation SSD**

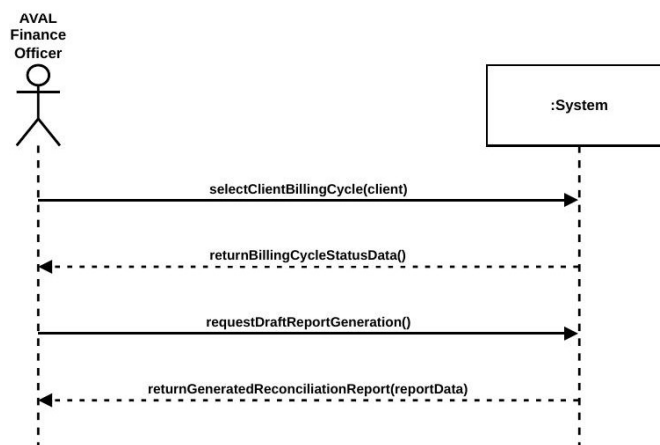
**UC10: Identify Financial Anomalies SSD**



UC11: Generate Verified Reconciliation Report SSD

Use Case 11

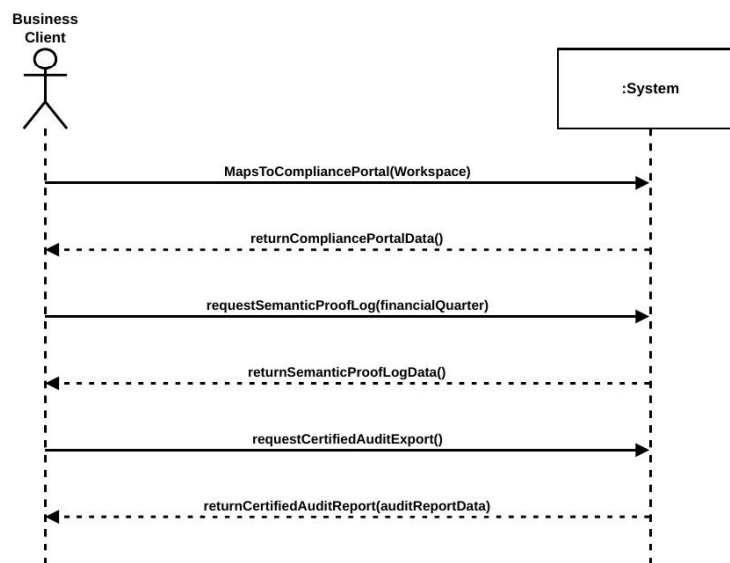
Generate Verified Reconciliation Report



UC12: Create and Maintain Verified Transaction History SSD

Use Case 12

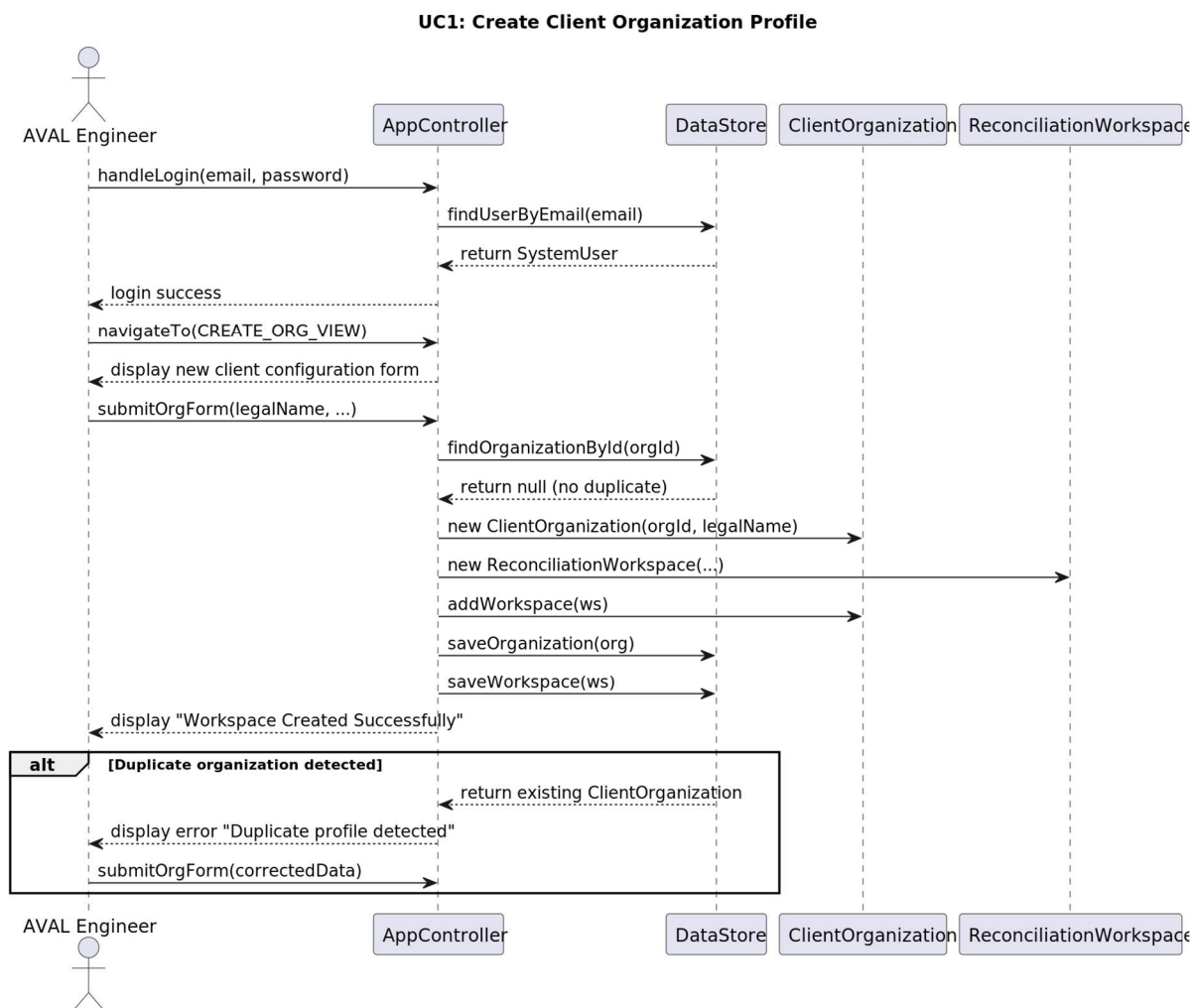
Create and Maintain Verified Transaction History



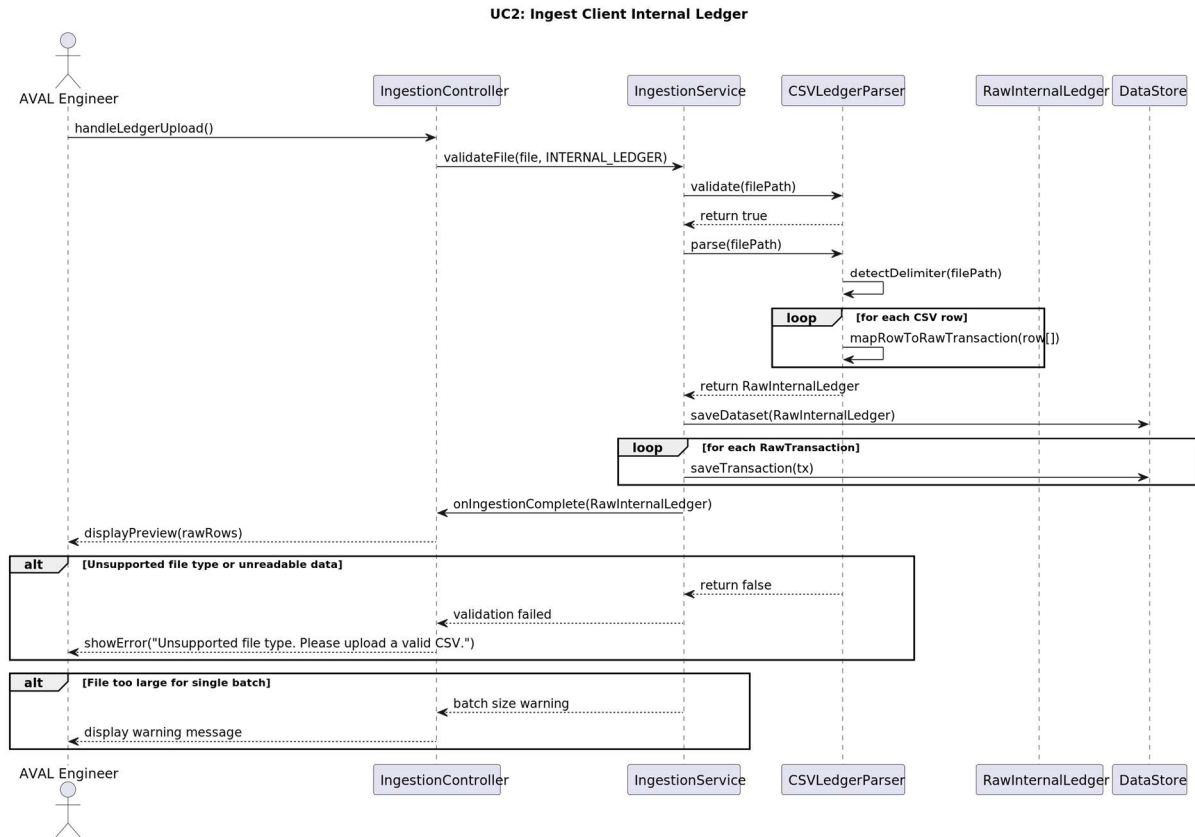
• Sequence Diagram

The following detailed Sequence Diagrams show the complete message flow between objects for each of the twelve use cases, including method calls, return values, and conditional paths. These diagrams correspond to the UC1-UC12 SVG artifacts in the docs/Sequence Diagrams/ folder of the AVAL project repository.

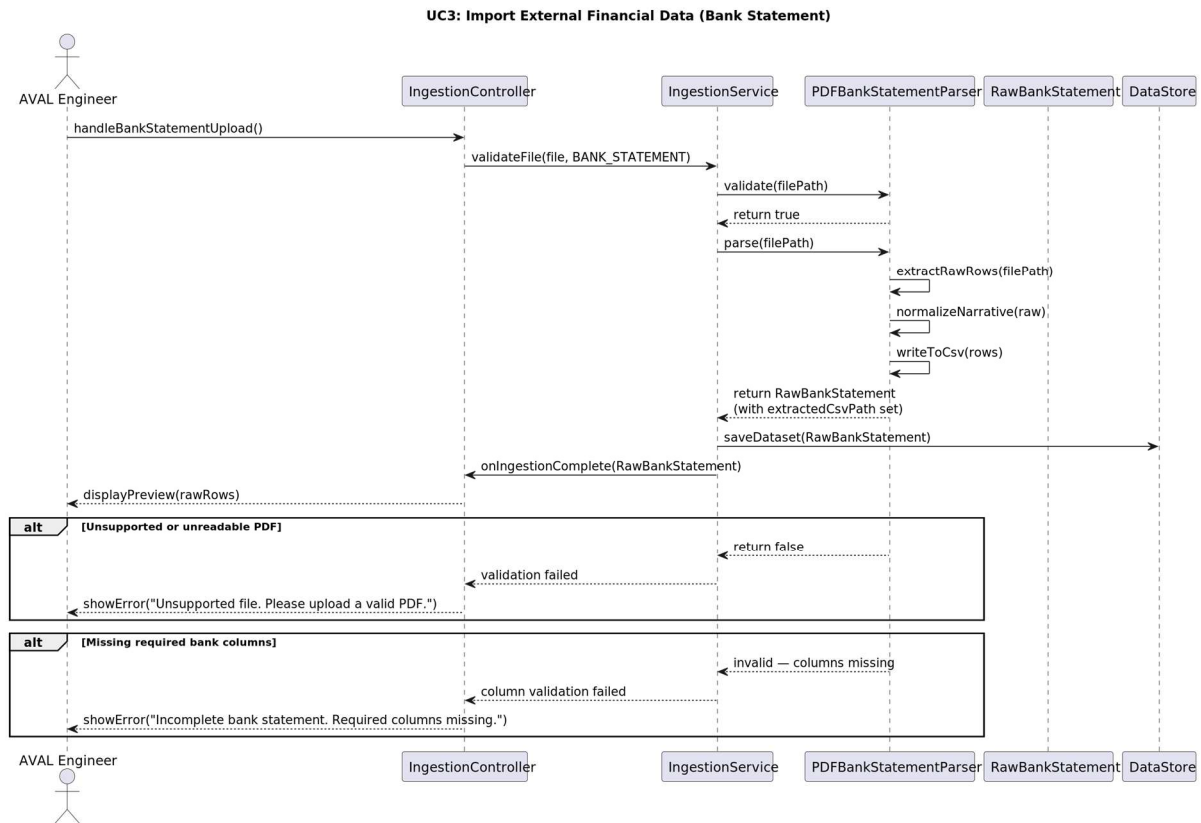
UC1: Create Client Organization Profile



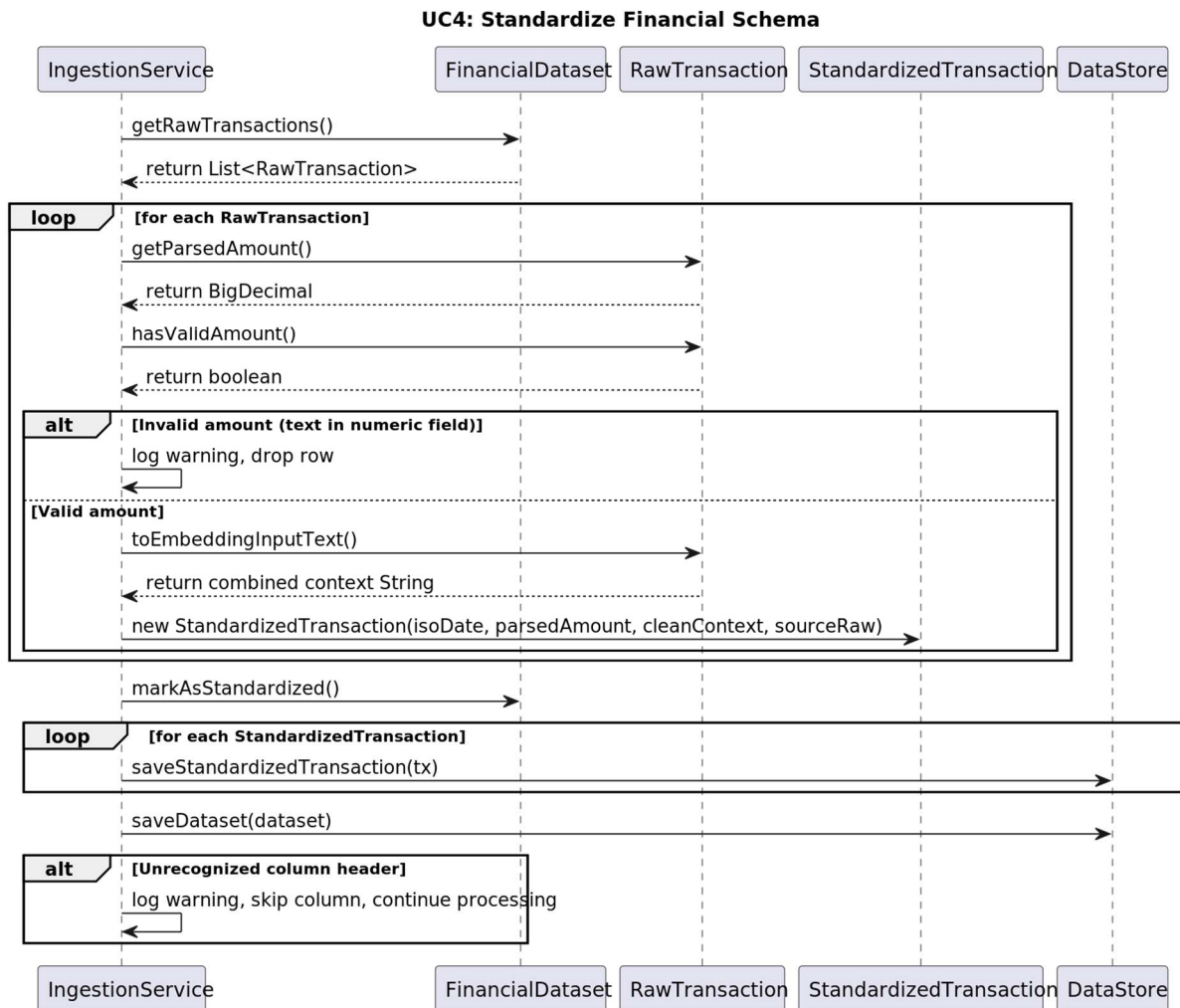
UC2: Ingest Client Internal Ledger



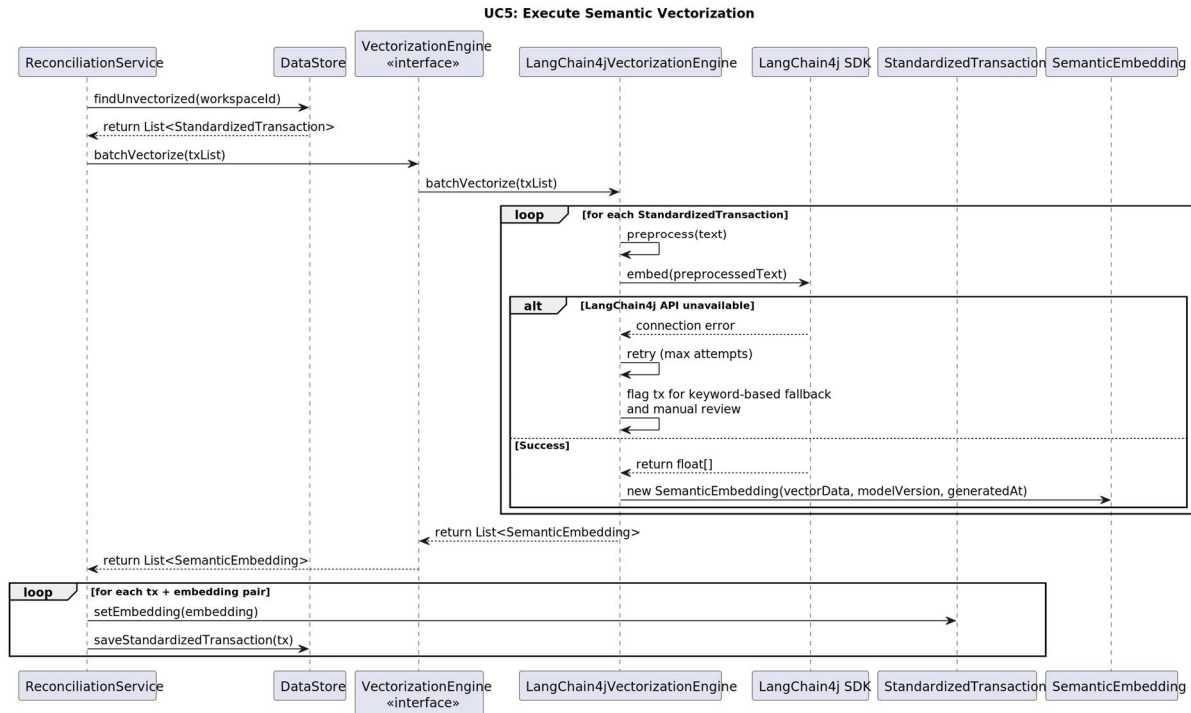
UC3: Import External Financial Data



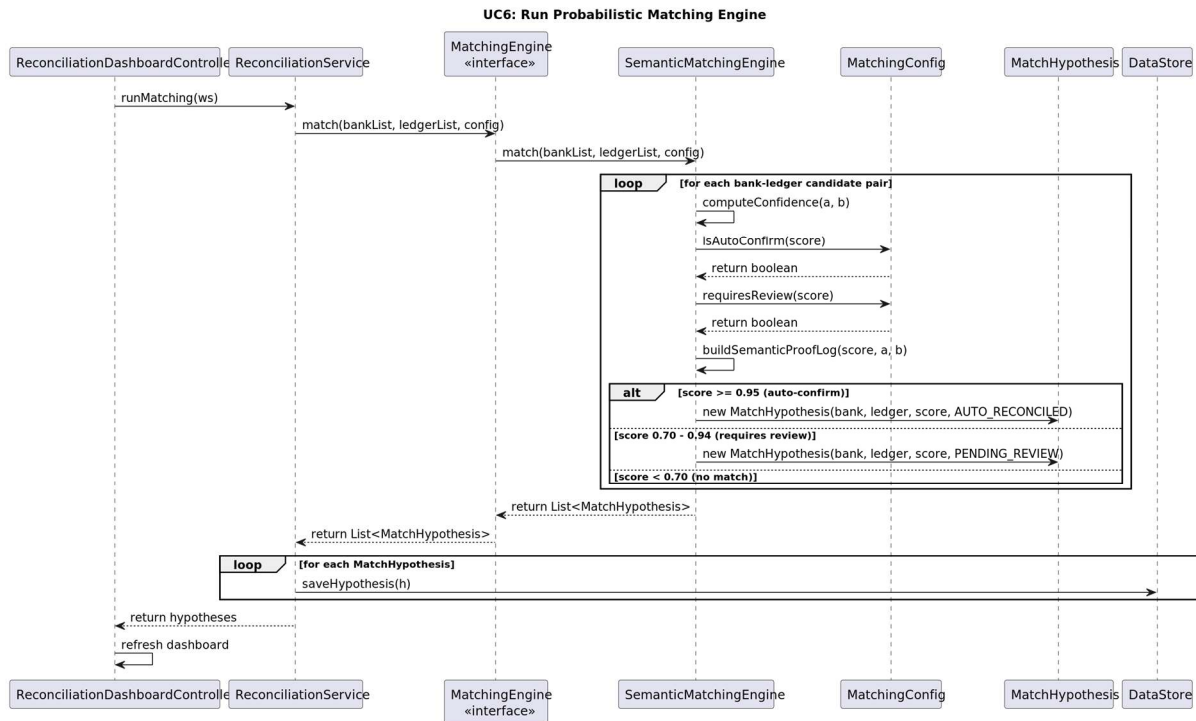
UC4: Standardize Financial Schema



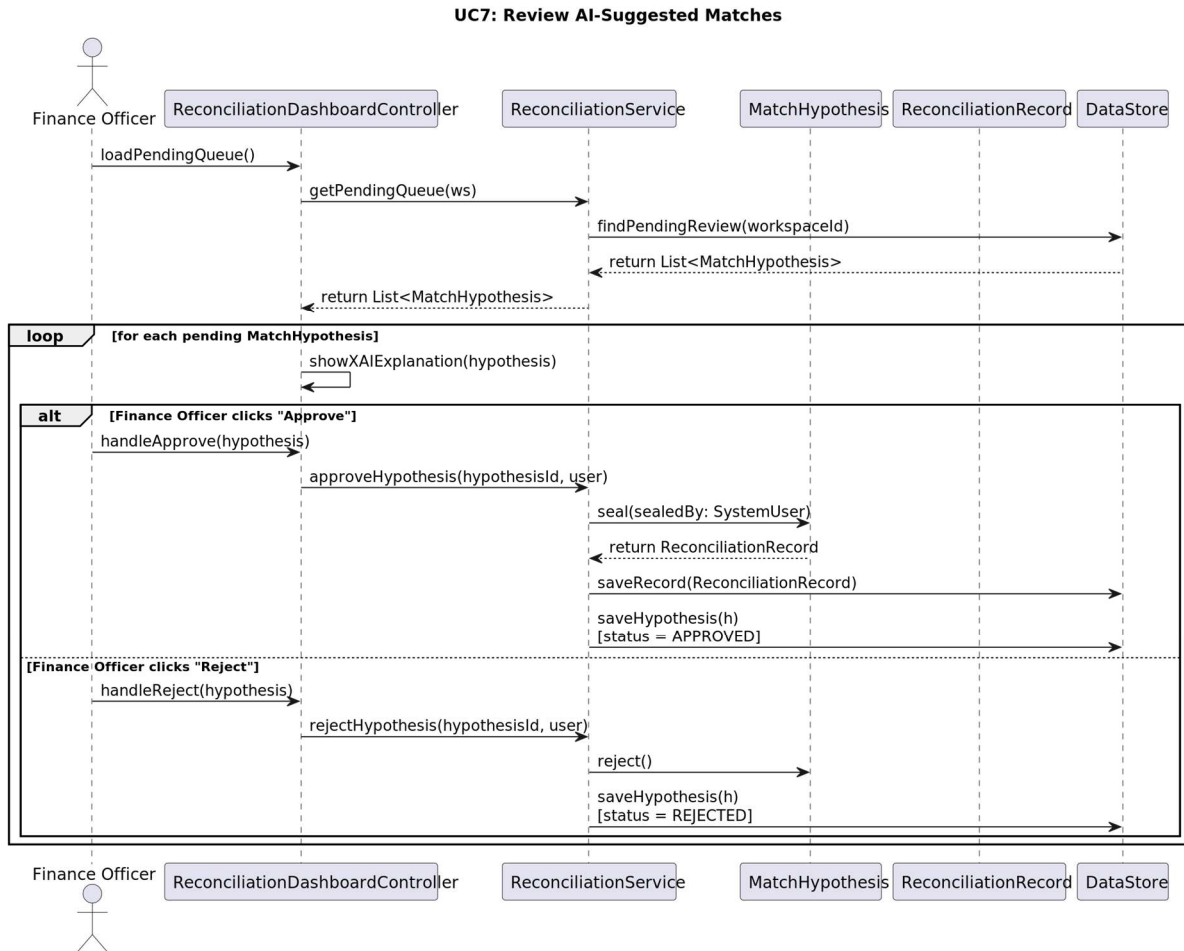
UC5: Execute Semantic Vectorization



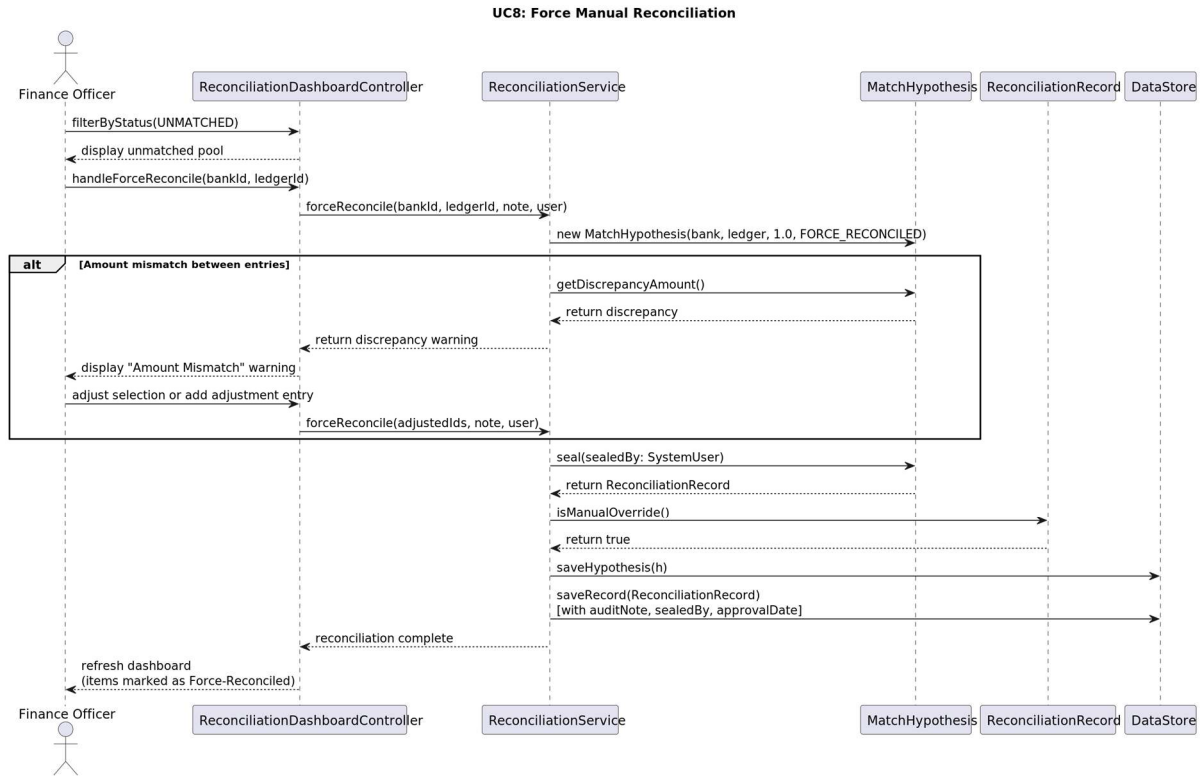
UC6: Run Probabilistic Matching Engine



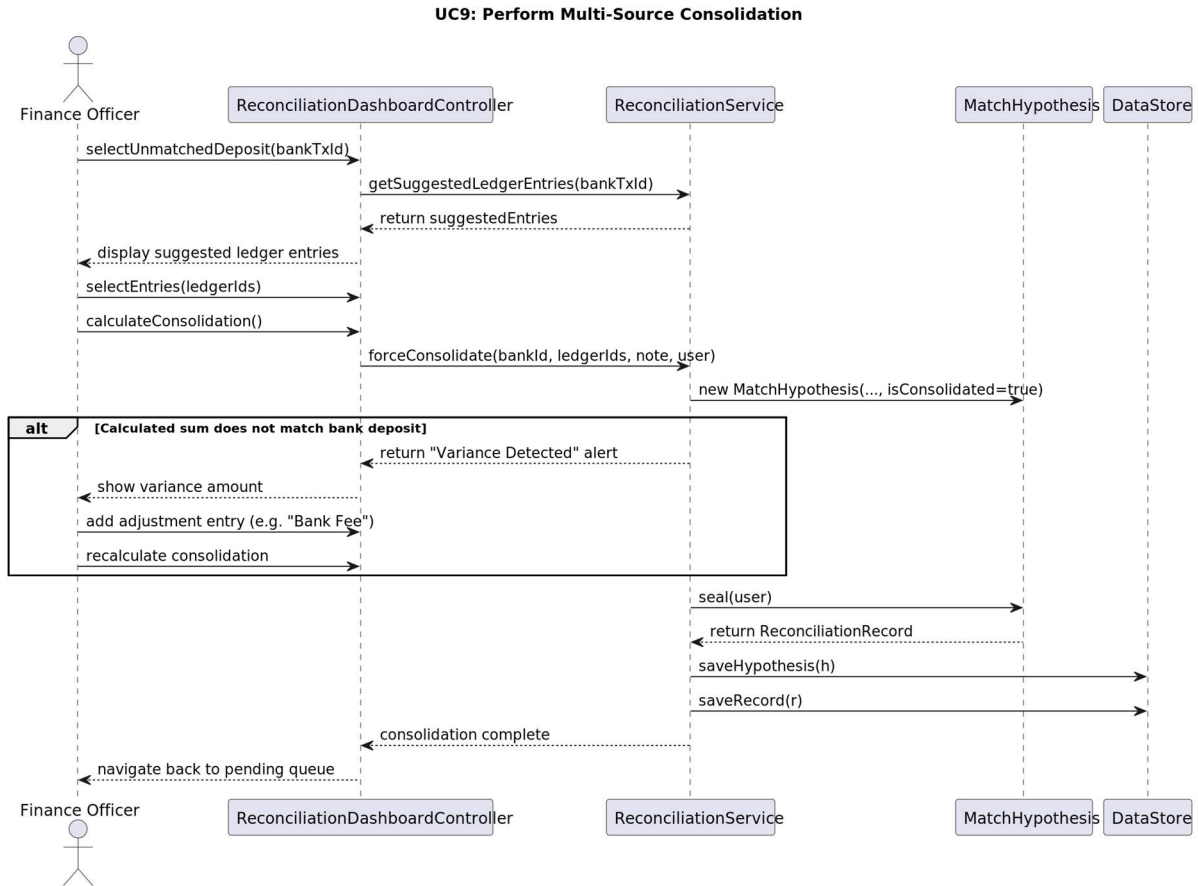
UC7: Review AI-Suggested Matches



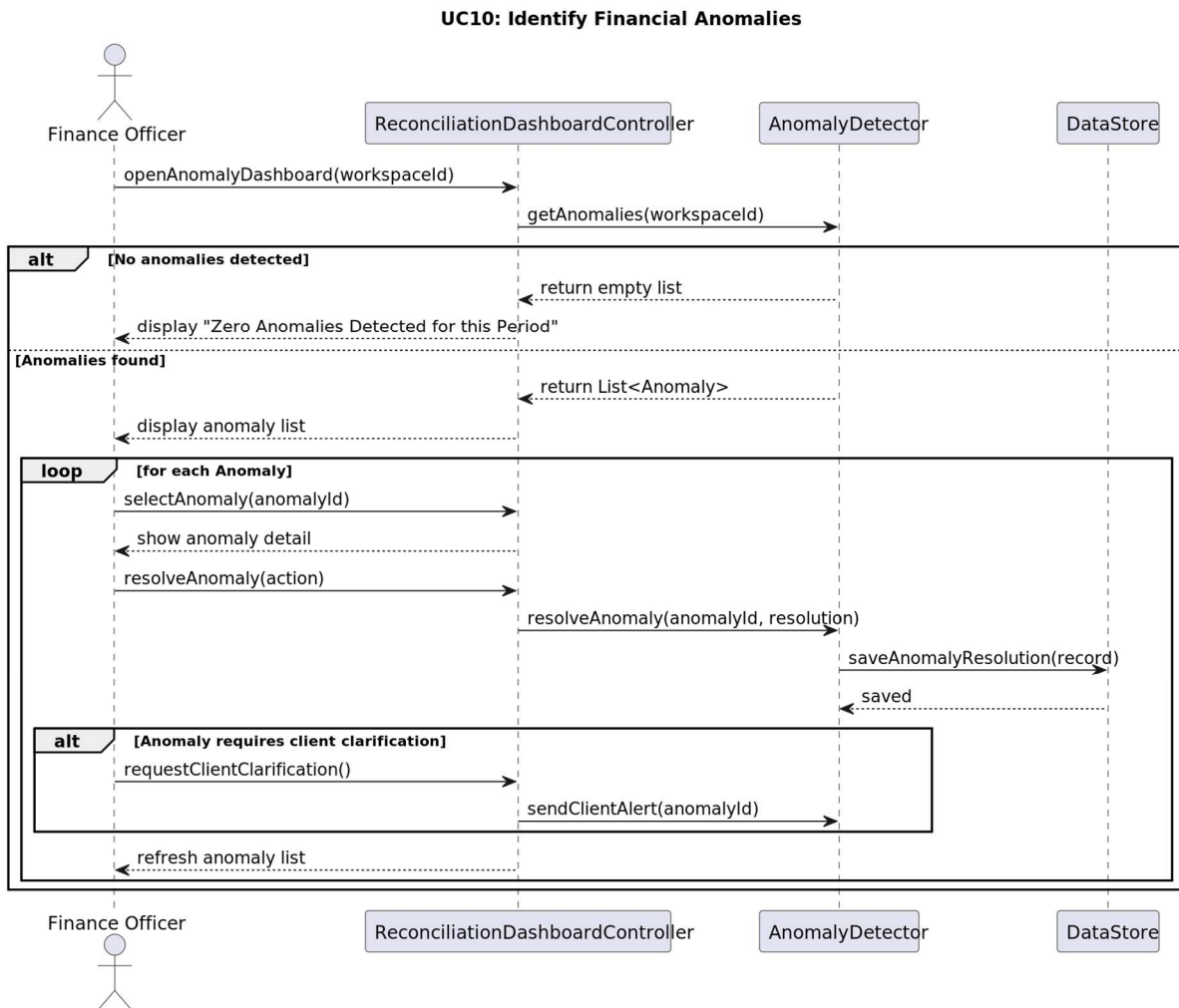
UC8: Force Manual Reconciliation



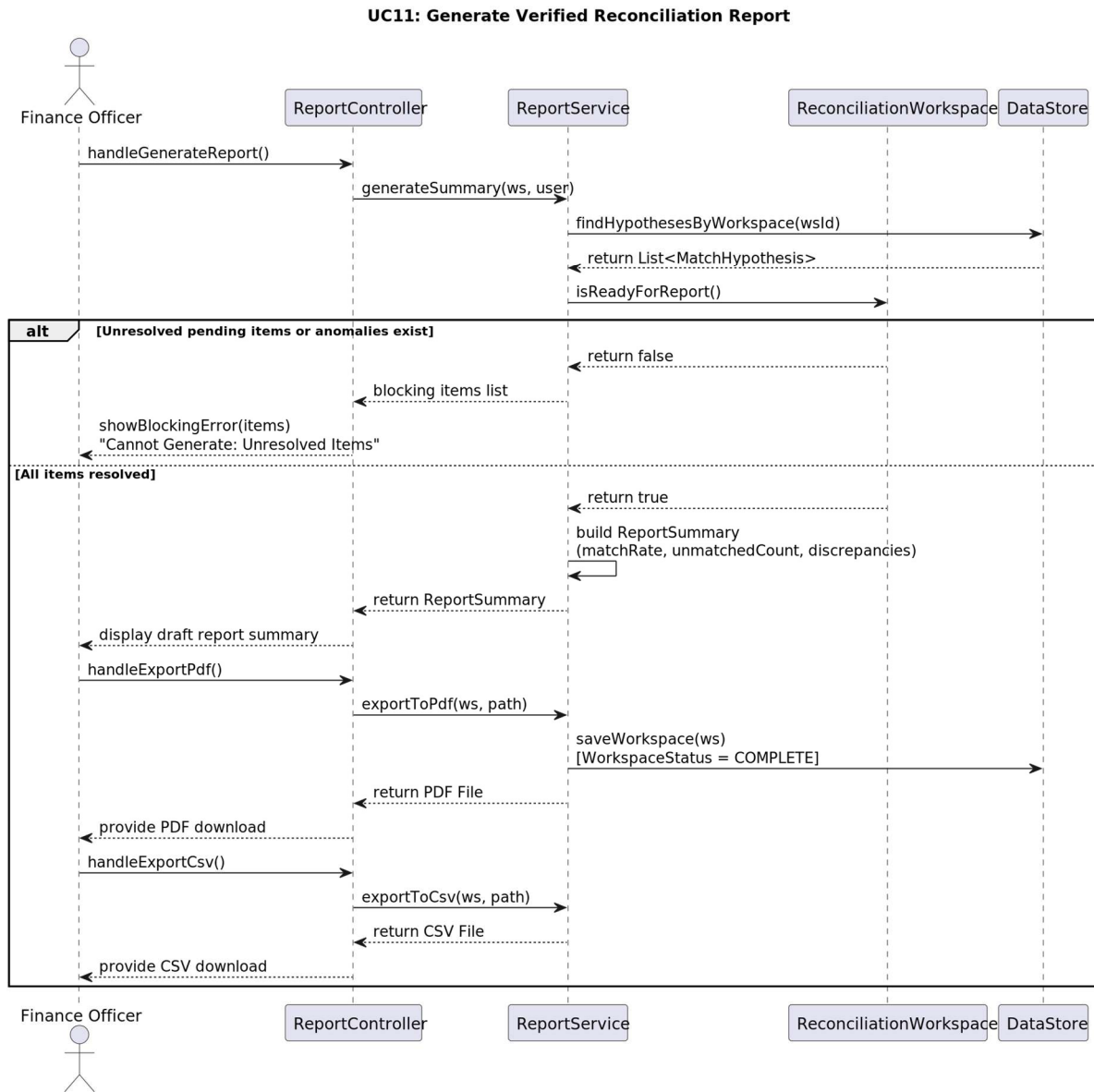
UC9: Perform Multi-Source Consolidation



UC10: Identify Financial Anomalies

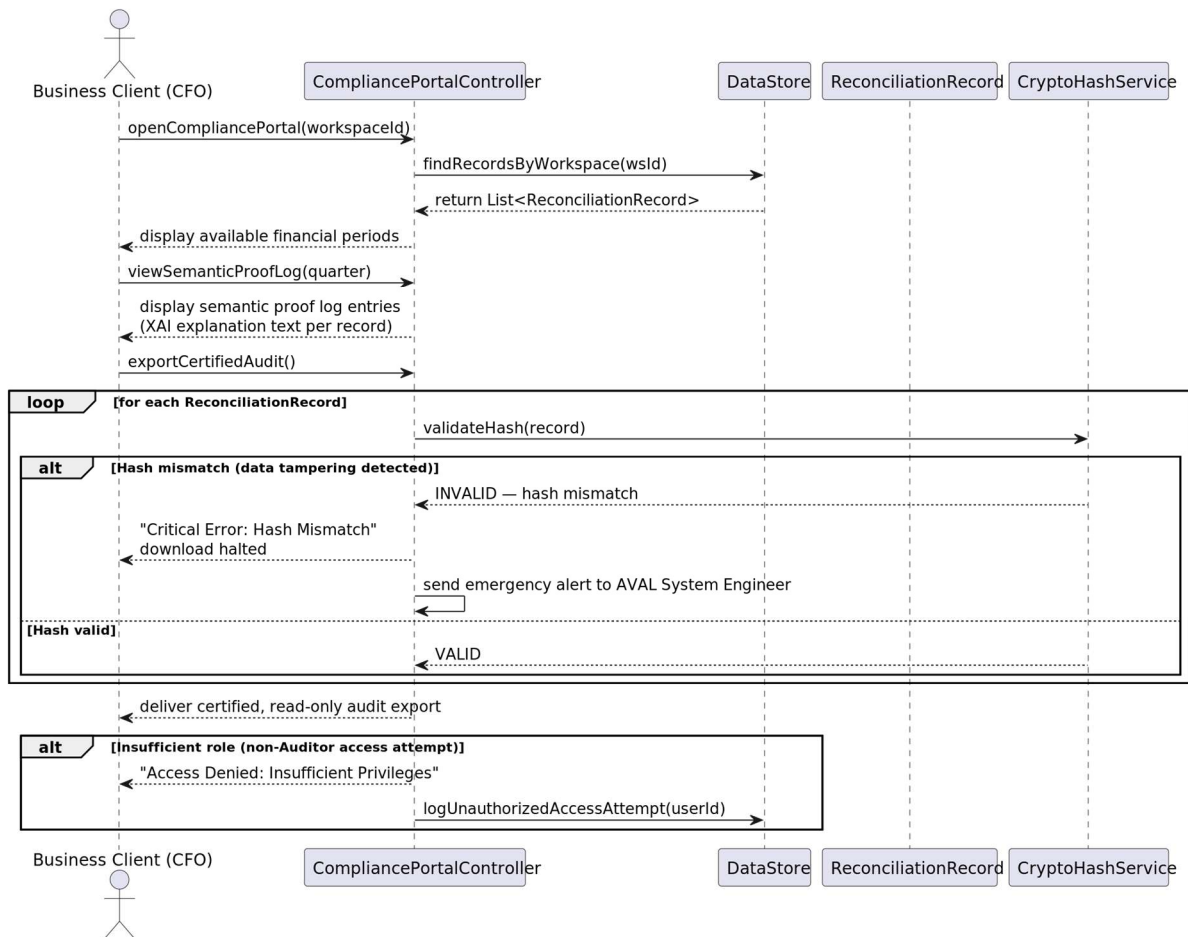


UC11: Generate Verified Reconciliation Report



UC12: Create and Maintain Verified Transaction History

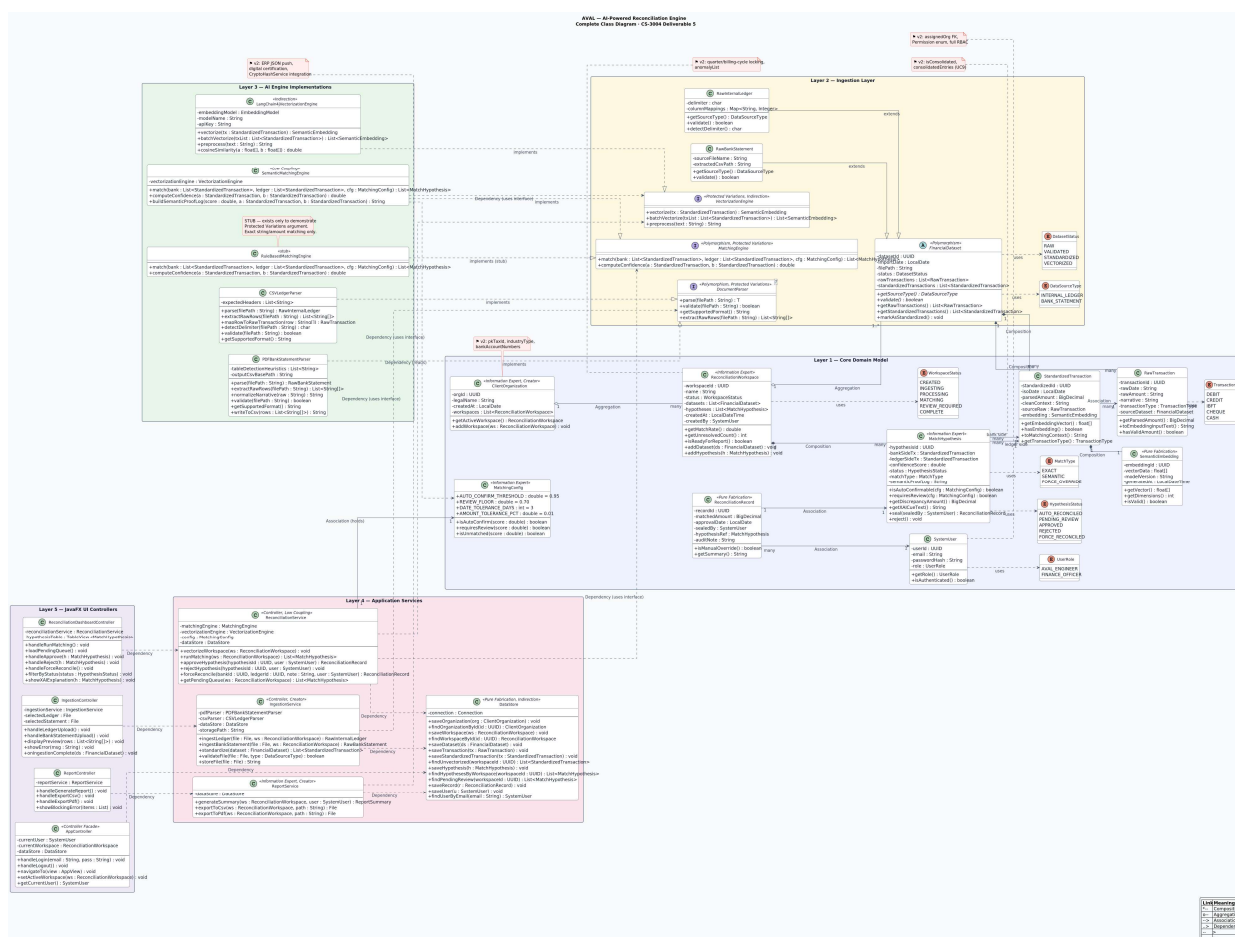
UC12: Create and Maintain Verified Transaction History



- **Class Diagram**

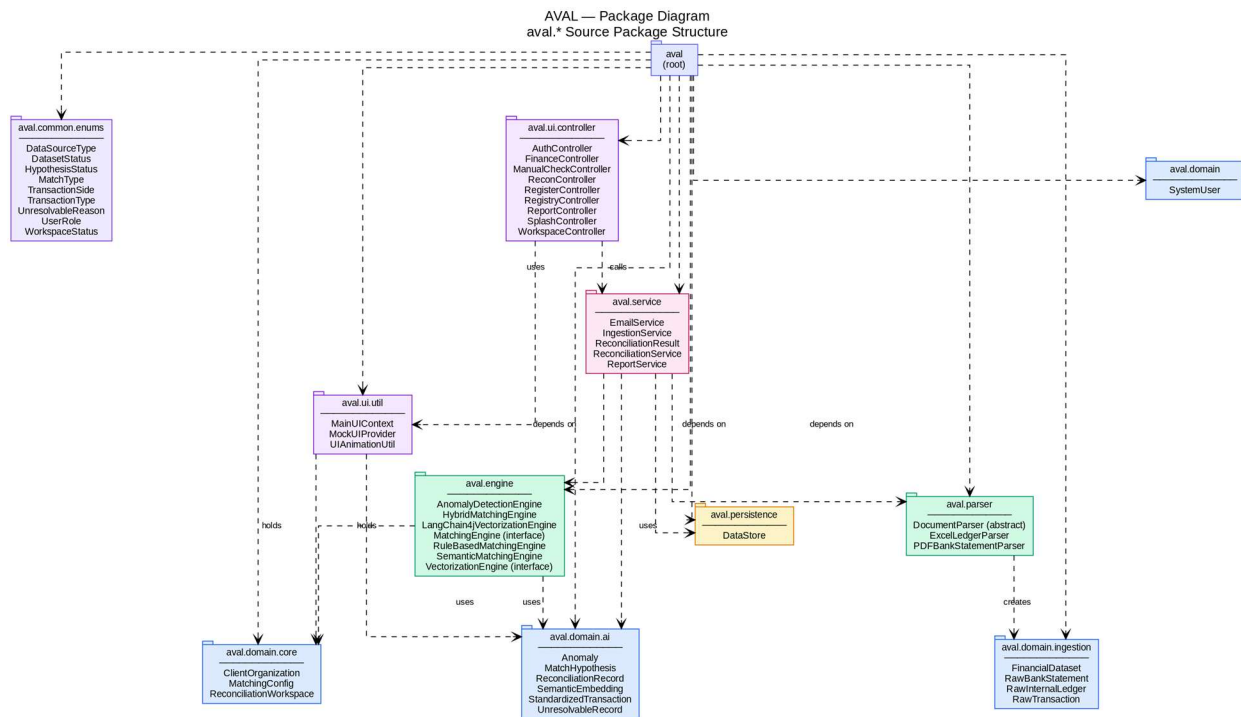
The Class Diagram below presents the complete five-layer architecture of AVAL. Layer 1 (Core Domain Model) contains the primary entities and enumerations. Layer 2 (Ingestion Layer) defines the interfaces and abstract classes. Layer 3 (AI Engine Implementations) contains the concrete matching and vectorization engines. Layer 4 (Application Services) defines the service facades. Layer 5 (JavaFX UI Controllers) contains the GRASP Controller classes. The source PlantUML definition is available in docs/AVAL Class Diagram.puml.

GRASP patterns applied: Information Expert, Creator, Controller, Pure Fabrication, Low Coupling, High Cohesion, Polymorphism, Protected Variations, Indirection. GoF patterns applied: Strategy, Template Method, Repository/DAO, Facade, Adapter, Factory Method.



● Package Diagram

The Package Diagram below illustrates the organisation of the AVAL source code into Java packages under the aval.* root namespace. Seven top-level packages are defined, each with a clear single responsibility. The dependencies between packages are shown with dashed arrows, demonstrating the layered dependency structure from UI → Services → Engines/Parsers → Domain → Common.



• Deployment Diagram

The Deployment Diagram below shows the physical deployment topology of AVAL on a developer's local machine. The system consists of: (1) a JVM process running the JavaFX 21 desktop application, (2) a Docker Compose network hosting three containers: the PostgreSQL 16 + pgvector database (aval-postgres), the Ollama inference server (aval-ollama), and a one-shot model downloader sidecar (aval-ollama-pull). Communication between the application and the database occurs over JDBC/HikariCP on port 5432. Communication with the Ollama embedding service occurs over HTTP REST on port 11434.

